

# Syllabus: Leveraging Large Language Models (LLMs) for Vulnerability Detection in Cybersecurity

---

**Author:** Badr TAJINI — LLMs for Vulnerability Detection in Cybersecurity — ECE — 2025/2026

**ECTS CREDITS : X**

**NUMBER OF CONTACT HOURS : 16H**

| LECTURES | LABS | PRACTICALS | PROJECTS |
|----------|------|------------|----------|
| 8        | 8    | 0          | 0        |

**NUMBER OF HOURS OF SELF-STUDY: ~20h for the final project**

**TEACHING MODE : Hybrid**

**LANGUAGE : English (LLM interactions and resources are often in English)**

## Course Objectives

- Understand the role and potential of LLMs in cybersecurity. Master LLM fundamentals that are relevant for security. Develop prompt-engineering skills for vulnerability detection. Apply LLMs to source-code and configuration analysis to identify potential flaws. Critically assess LLM outputs and understand their intrinsic limitations. Use LLMs ethically and responsibly in a security context (privacy, bias).

## Prerequisites

- Foundational knowledge in cybersecurity (common vulnerability classes, e.g., OWASP Top 10).
- Basic programming/scripting concepts (reading and understanding code — e.g., Python, JavaScript, Java).
- Interest in artificial intelligence and its security applications.

## Content

The course includes the following theoretical sessions (8 modules of about 1 hour each):

1. **Module 1:** Introduction — AI & LLMs in the cybersecurity landscape
2. **Module 2:** LLM fundamentals for cybersecurity engineers
3. **Module 3:** The craft of prompt engineering for vulnerability detection
4. **Module 4:** Applying LLMs to source-code analysis (SAST augmentation)
5. **Module 5:** LLMs for configuration review & IaC security (Infrastructure as Code)
6. **Module 6:** Beyond code & configurations — other potential applications
7. **Module 7:** Evaluating LLM results & the human-in-the-loop imperative
8. **Module 8:** Limitations, LLM-related security risks, and future trends

Note : **Mini-projects** (8h total; 4 × 2h): hands-on practice applying the above (getting started with LLMs, prompting for code/config, critical analysis of outputs).

## Skills to Acquire

1. Explain the capabilities and limitations of LLMs for security analysis tasks.
2. Formulate clear, specific, and contextual prompts to query an LLM on cybersecurity questions.
3. Use an LLM as an assistant to identify potential vulnerabilities in code snippets (e.g., Python, JavaScript, Java).
4. Use an LLM to look for common misconfigurations in configuration files (e.g., SSH, Nginx, cloud policies).
5. Critically analyze and verify security suggestions or vulnerabilities flagged by an LLM.
6. Understand data-privacy risks when using LLMs with sensitive code or configurations.
7. Write a concise security-review report based on an LLM's results, including critical human analysis.

## Assessment Methods

- **Continuous assessment (graded mini-projects)** — 50%
- **Final project** — 50%: an LLM-assisted security review of a provided artifact, with a detailed report (analyze/critique LLM interactions).

## Bibliographic References

- [1] Pearce, H., Ahmad, B., Tan, B., Dolan-Gavitt, B., & Karri, R. (2022). *Asleep at the Keyboard? Assessing the Security of GitHub Copilot's Code Contributions*. In IEEE Symposium on Security and Privacy (SP) 2022.
- [2] Noever, D. (2023). *Can Large Language Models Find And Fix Vulnerabilities? A Case Study on GitHub Copilot*. arXiv:2304.01467.
- [3] Fu, T., Chen, J., Li, W., Wang, H., Jin, H., & Zhang, H. (2023). *Does GPT-4 Pass the Bar Exam on Cybersecurity? Evaluating GPT-4's Security Capabilities*. arXiv:2312.11341.
- [4] Sobania, D., Briesch, M., Hanna, C., & Petke, J. (2023). *An Analysis of the Automatic Bug Fixing Performance of ChatGPT*. arXiv:2301.08653.
- [5] Schuster, E., Adolphs, L., Lammich, P., & Neumann, G. (2023). *Can we trust LLMs? An Assessment of Code Generation Capabilities of ChatGPT*. arXiv:2307.08164.
- [6] Fischer, M., Zipf, S., & Krombholz, K. (2024). *Large Language Models for Vulnerability Detection: A Systematic Literature Review*. arXiv:2402.15416.
- [7] Oliynykov, R. (2023). *Large Language Models for Source Code Analysis*. Vector Institute.
- [8] Perry, N., Simko, M., & Aharoni, M. (2023). *Security Implications of Large Language Models*. IBM Research.
- [9] Happe, A., & Siemens Cybersecurity Team. (2023). *Using Large Language Models (LLMs) for cybersecurity: perspectives, benefits, and risks*. Siemens Whitepaper.
- [10] Kang, D., Li, X., Sims, C., Appel, A. W., & Wang, T. (2023). *Exploring the Security Vulnerabilities of Large Language Models*. arXiv:2311.12513.

[11] Jones, N. (2023). *How AI is shaking up code generation — and cybersecurity*. Nature News Feature.

[12] Dakhel, A. M., Rauf, U., Eltanbouly, S., Malik, H., & Nandi, A. (2023). *Leveraging Large Language Models for Automated Unit Test Generation and Vulnerability Detection*. In Proc. 2023 ACM/IEEE AST.

## Additional Resources

- **OWASP Top 10 for Large Language Model Applications:** OWASP project dedicated to risks specific to LLM-based applications. Useful for understanding attack vectors against LLM applications themselves.  
(<https://owasp.org/www-project-top-10-for-large-language-model-applications/>)
  - **Technical blogs** from LLM providers (OpenAI, Google AI, Anthropic) and security companies (NCC Group, Trail of Bits, Synack, etc.): frequent posts on capabilities, limitations, and secure usage of their models or LLMs in general.
- 

## Annex — Course Roadmap (Lectures)

### Module 1: Introduction — AI & LLMs in the cybersecurity landscape (1h)

- The evolving role of AI in cybersecurity (detection, response, threat intelligence).
- What are LLMs? High-level overview (Transformers, tokens, training data). No heavy math; focus on security-relevant conceptual understanding.
- Why use LLMs for vulnerability detection? Potential benefits (speed, scale, new perspectives).
- Opportunities vs hype: set realistic expectations.
- Ethics & responsible use: data privacy (critical when entering code/configurations), bias, potential misuse.
- Overview of course structure and objectives.

### Module 2: LLM fundamentals for cybersecurity engineers (1h)

- Core LLM concepts: prompting, context window, temperature, tokens, embeddings (simplified).
- Access/usage models: general-purpose models (GPT series, Claude, Llama, Bard) vs. potentially security-specialized models (if available/relevant). API access vs. Web interface.
- Strengths in a security context: natural-language understanding, varying degrees of code understanding, pattern recognition, summarization.
- Weaknesses & risks: hallucinations (false positives/negatives), lack of true understanding/reasoning, exposure to prompt injection, data poisoning, dependence on training-data quality and freshness.

### Module 3: The craft of prompt engineering for vulnerability detection (1.5h)

- Principles of effective prompting: clarity, context, specificity, role-play (“Act as a security code auditor…”).
- Zero-shot vs few-shot prompting: include examples in the prompt.
- Structuring prompts for security tasks:
  - Provide code snippets/files effectively.
  - Ask for specific vulnerability classes (e.g., “Check this Python code for possible SQL injection”).
  - Request explanations, severity assessments (with caution), or potential remediation advice.
  - Specify output formats (e.g., “List findings with CWE identifiers”).

- Iterative refinement of prompts: how to improve prompts based on LLM responses.

#### **Module 4: Applying LLMs to source-code analysis (SAST augmentation) (1.5h)**

- Use LLMs to identify common code vulnerabilities (focus on OWASP Top 10: SQLi, XSS, insecure deserialization, command injection, etc.).
- Language specificity: how LLM performance varies by language (e.g., Python, Java, JavaScript, C#).
- Comparison with traditional SAST tools: strengths (NL queries, potential logical-bug spotting) vs weaknesses (consistency, completeness, FP/FN, scalability challenges).
- Case studies/examples: demonstrations of successful (and unsuccessful) vulnerability identification on code snippets.

#### **Module 5: LLMs for configuration review & IaC security (1h)**

- Analyze configuration files for common issues (web servers, databases, SSH, cloud services — IAM, S3 buckets).
- Identify insecure defaults, overly permissive settings, missing security controls.
- Apply to Infrastructure as Code: review Terraform, CloudFormation, etc., for security issues before deployment.
- Prompting strategies specific to configuration analysis.

#### **Module 6: Beyond code & configurations — other potential applications (0.5h)**

- Analyze security documentation, policies, and threat-intel reports (summarization, entity extraction).
- Assist in interpreting vulnerability-scan or pentest outputs.
- **Caution note:** emphasize that limitations and risks increase as the task becomes less structured than code/config review. High-level glimpses on textual descriptions of network traffic/logs (not deep packet inspection).

#### **Module 7: Evaluating LLM results & the human-in-the-loop imperative (1h)**

- Verification techniques for LLM findings: manual review, corroboration with other tools, targeted testing (where applicable).
- Understand and manage false positives & false negatives.
- Calibrating confidence: LLMs often sound confident even when wrong — why human expertise remains essential.
- Build a workflow that integrates LLMs as assistants, not replacements; counter over-reliance.

#### **Module 8: Limitations, LLM-related security risks, and future trends (0.5h)**

- Recap of key limitations (hallucinations, context windows, training-data cutoff, privacy).
- User-side security risks: prompt injection (leakage of sensitive information in prompts), data-privacy violations.
- Adversarial attacks against LLMs (brief overview).
- Outlook: security-specialized LLMs, integrations into IDEs and security platforms (e.g., GitHub Copilot security features), ongoing research in the community.